

A) For  $M_1$

Create matrix  $M[n][n]$

for  $i \leftarrow 0$  to  $n-1$  do

for  $j \leftarrow 0$  to  $n-1$  do

$M[i][j] \leftarrow 5 + n * i + j$

↳ from equation derived from the matrix

$$M[x, y] = 5nx + y$$

B) For  $M_2$

Create matrix  $M[n][n]$

value  $\leftarrow 5$

for diagonal  $d \leftarrow 0$  to  $2n-2$  do

if  $d$  is even then

row  $\leftarrow \min(d, n-1)$

col  $\leftarrow d - \text{row}$

while row  $\geq 0$  AND col  $< n$  do

$M[\text{row}][\text{col}] \leftarrow \text{value}$

value  $\leftarrow \text{value} + 1$

row  $\leftarrow \text{row} - 1$

col  $\leftarrow \text{col} + 1$

Else

col  $\leftarrow \min(d, n-1)$

row  $\leftarrow d - \text{col}$

while col  $\geq 0$  AND row  $< n$  do

$M[\text{row}][\text{col}] \leftarrow \text{value}$

value  $\leftarrow \text{value} + 1$

row  $\leftarrow \text{row} + 1$

col  $\leftarrow \text{col} - 1$

3) For M3

Create matrix  $M[n][n]$   
for  $i \leftarrow 0$  to  $n-1$  do  
for  $j \leftarrow 0$  to  $n-1$  do  
 $M[i][j] \leftarrow 5 + i + n * j$   
from the equation

$$M(x, y) = 5 + x + ny$$

B) Algorithm SearchSS (M, key)

$i \leftarrow 0$

$j \leftarrow n-1$

while  $i < n$  AND  $j \geq 0$  do

if  $M[i][j] = \text{key}$  then

print (i, j)

break

Else if key  $< M[i][j]$  then

$j \leftarrow j-1$

Else

$i \leftarrow i+1$

print "Not Found"

B1) Time Complexity

- One Comparison per step which each step eliminates one row or one column

Worst Case  $2n-1$  Comparisons

$\rightarrow \underline{O(n)}$

B2) Space Complexity

\_\_\_\_\_

C1) Algorithm  $\text{DACSearchSS}(M, \text{key}, \text{top}, \text{bottom}, \text{left}, \text{right})$

if  $\text{top} > \text{bottom}$  OR  $\text{left} > \text{right}$  then  
return not found

$\text{midRow} \leftarrow (\text{top} + \text{bottom}) / 2$

$\text{midCol} \leftarrow (\text{left} + \text{right}) / 2$

if  $M[\text{midRow}][\text{midCol}] = \text{key}$  then  
Print  $(\text{midRow}, \text{midCol})$   
return

else if  $\text{key} < M[\text{midRow}][\text{midCol}]$  then

$\text{DACSearchSS}(M, \text{key}, \text{top}, \text{midRow}-1, \text{left}, \text{midCol}-1)$

$\text{DACSearchSS}(M, \text{key}, \text{top}, \text{midRow}-1, \text{midCol}, \text{right})$

$\text{DACSearchSS}(M, \text{key}, \text{midRow}, \text{bottom}, \text{left}, \text{midCol}-1)$

else

$\text{DACSearchSS}(M, \text{key}, \text{top}, \text{midRow}, \text{midCol}+1, \text{right})$

$\text{DACSearchSS}(M, \text{key}, \text{midRow}+1, \text{bottom}, \text{left}, \text{midCol})$

$\text{DACSearchSS}(M, \text{key}, \text{midRow}+1, \text{bottom}, \text{midCol}+1, \text{right})$

C1) Time-Complexity

Worst Case recurrence check the three quadrants

$$T(n) = 3T(n/2) + 1$$

Applying master theorem  $a=3, b=2, f(n)=1$

$$\text{result} = O(n^{\log_2 3})$$

C2) Space-Complexity

D1) Mathematical Comparison

$\text{SearchSS} \rightarrow O(n)$  Whereas  $\text{DACSearchSS} > O(n)$

Therefore  $\text{SearchSS}$  is faster than  $\text{DACSearchSS}$